

ПРАВИТЕЛЬСТВО РОССИЙСКОЙ ФЕДЕРАЦИИ
ФГАОУ ВО НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ
«ВЫСШАЯ ШКОЛА ЭКОНОМИКИ»

Факультет компьютерных наук
Образовательная программа «Прикладная математика и информатика»

УДК 004.75

Отчет об исследовательском проекте на тему:
Системы балансировки нагрузки в гибридных распределенных системах

Выполнил студент:

группы #БПМИ236, 3 курса

Разухин Александр Сергеевич

Принял руководитель проекта:

Курочкин Илья Ильич
к.т.н., зав.лаб. Ц-1
ИППИ РАН

Москва 2026

Содержание

Аннотация	3
1 Введение	4
1.1 Описание предметной области	4
1.2 Релевантность и значимость	4
1.3 Цели и задачи	5
1.4 Метрики оценки производительности	5
1.5 Предполагаемые результаты проекта	6
2 Литературный обзор	6
2.1 Обзор предметной области	6
2.2 Обзор существующих методов	8
2.3 Особенности BOINC	9
2.4 Сравнение подходов	9
2.5 Выводы из обзора	9
Список литературы	11

Аннотация

При проектировании высокопроизводительных и распределенных систем одной из важнейших задач является выбор и настройка системы выдачи заданий (балансировщик нагрузки). При использовании гомогенных вычислительных узлов в кластере можно использовать готовые решения (slurm, torque и др.), но в гетерогенных системах готовые решения не будут работать из коробки. В распределенных системах из-за большой гетерогенности и других ограничений может возникнуть потребность разработки оригинального планировщика заданий. В распределенной вычислительной системе в качестве части ресурсов могут быть использованы облачные ресурсы или ресурсы кластеров. Уже были разработаны решения (к примеру, CluBoRun в грид-системе на платформе BOINC) по использованию ресурсов кластера в фоновом режиме. Предлагается разработать /модифицировать систему выдачи заданий для грид-системы на платформе BOINC, в которой присутствуют сегменты кластеров.

Ключевые слова

Распределенные системы, балансировка нагрузки

1 Введение

1.1 Описание предметной области

Предметная область проекта охватывает распределенные системы с гетерогенной инфраструктурой с основными компонентами:

- BOINC (Berkeley Open Infrastructure for Network Computing) — платформа для добровольных вычислений, объединяющая ресурсы разнородных устройств (серверов, кластеров, персональных компьютеров) в единую сеть
- Кластеры — узлы, интегрированные в распределенную систему с целью повышения доступности
- Гетерогенные ресурсы — устройства с различными характеристиками: вычислительной мощностью, объемом памяти, доступностью и другими
- Балансировщик нагрузки — компонент, отвечающий за распределение вычислительных задач между узлами с учетом их возможностей и текущих состояний

Также предметная область имеет следующие особенности:

- Отсутствие централизованного контроля за узлами системы
- Разнородность ресурсов каждого из узлов
- Различные режимы работы узлов (не все устройства доступны на протяжении 100% времени)
- Различный объем задач, время выполнения которых может варьироваться от нескольких минут до нескольких дней
- Необходимость минимизации накладных расходов для балансировщика

1.2 Релевантность и значимость

Актуальность данного проекта обусловлена множеством существующих на данный момент проблем, таких как:

- 1 Современные вычислительные задачи такие, как анализ больших данных, биоинформатика, требуют огромного количества ресурсов, что способствует росту числа гетерогенных распределенных систем

- 2 Стандартные планировщики задач оптимизированы под гомогенные системы и не учитывают различия в производительности узлов и их доступность, что в гетерогенных системах приводит к простаиванию некоторых узлов или, наоборот, излишней загруженности отдельных
- 3 Необходимость развития динамических алгоритмов балансировки в противовес существующим решениям с их статическими правилами
- 4 Оптимизация всех вычислений, выполняющихся на платформе BOINC

1.3 Цели и задачи

Целью данного проекта является разработка системы выдачи заданий для распределенной системы на платформе BOINC, в которой присутствуют сегменты кластеров. Для ее достижения необходимо было выполнить следующие задачи:

- 1 Обзор систем балансировки нагрузки для кластеров и распределенных систем
- 2 Выбор системы моделирования и адаптация модели под специфику гибридной распределенной системы
- 3 Реализация/модификация системы балансировки нагрузки гибридной распределенной системы
- 4 Тестирование системы балансировки на модели гибридной распределенной системы

1.4 Метрики оценки производительности

Выбор метрик обусловлен необходимостью учета как общих показателей производительности, так и особенностей гетерогенной вычислительной среды.

- Пропускная способность (Throughput) – число выполненных задач за единицу времени. Формула: $T = \frac{N_{\text{completed}}}{\Delta t}$.
- Время выполнения задачи – среднее, медиана, перцентили (95%, 99%). Для каждой задачи i : $R_i = t_{\text{finish}}^{(i)} - t_{\text{arrival}}^{(i)}$. Среднее: $\bar{R} = \frac{1}{N} \sum_i R_i$.
- Загруженность узлов – для узла j : $u_j = \frac{\text{busy_cpu_time}_j}{\text{observation_time}}$. Сводные метрики: среднее по узлам $\bar{u}$ и коэффициент вариации $CoV = \frac{\sigma(u_j)}{\bar{u}}$ (показывает балансировку).

- Общее время завершения набора задач – $C = \max_i \{t_{\text{finish}}\} - \min_i \{t_{\text{arrival}}\}$
- Накладные расходы планировщика – среднее время, затраченное на принятие решения по задаче, и доля от общего времени:

$$OH_{\text{avg}} = \frac{1}{N} \sum_i t_{\text{sched}}^{(i)}, \quad OH\% = \frac{\sum_i t_{\text{sched}}^{(i)}}{T_{\text{wall}}} \cdot 100\%.$$

Такой набор метрик позволяет оценить как производительность системы, так и качество балансировки нагрузки в условиях гетерогенной среды.

1.5 Предполагаемые результаты проекта

- 1 Подробное описание методики моделирования гибридной распределенной системы
- 2 Подробное описание системы балансировки нагрузки гибридной распределенной системы
- 3 Анализ результатов моделирования системы
- 4 Доклад о проекте на научной конференции
- 5 Написание и публикация по итогам проделанной работы

2 Литературный обзор

2.1 Обзор предметной области

В своем начале Интернет имел очень мало потребителей, так что любой сервер мог выдерживать нагрузку даже самого популярного сайта, к тому же его падение не стало бы большой проблемой. Со временем люди были вынуждены заняться вопросами масштабирования ввиду растущей нагрузки. И если сначала проблема решалась увеличением числа процессоров, их улучшением, увеличением объема оперативной памяти, то после стало понятно, что есть потолок такого вертикального масштабирования, так что появилось понятие балансировки нагрузки. [2]

В общем случае, порядок работы балансировщика такой:

- 1 Получение входящих запросов от различных клиентов
- 2 Расчет размера входящих запросов и формирование очереди запросов
- 3 Периодическая проверка состояния серверов в пуле с помощью системы мониторинга

- 4 Использование стратегии / алгоритма / эвристики для выбора подходящего сервера под каждый входящий запрос

Сама идея балансировки нагрузки вносит в систему большое количество преимуществ:

- 1 Упрощенное отслеживания трафика
- 2 Улучшенное использование ресурсов
- 3 Балансировка нагрузки на сеть на основании загруженности серверов
- 4 Улучшенная доступность ресурсов
- 5 Уменьшение избыточного потребления ресурсов

[4]

Решения проблемы балансировки нагрузки делятся на 2 основных подхода в зависимости от того, принимает ли алгоритм решения на основе текущего состояния системы или нет: статический и динамический.

Статический подход предполагает наличие знаний о статусе системы, требованиях к ресурсам, времени связи и другим показателям. Балансировка при таком подходе достигается путем создания связей между списком процессоров и списком задач таким образом, чтобы достигалась максимальная производительность. Хотя назначения могут быть как детерминированными, так и вероятностными, текущее состояние системы не учитывается ни в одном из вариантов организации балансировки. Это существенный недостаток статических подходов, который оказывает существенное влияние на общую производительность из-за непредсказуемости колебаний нагрузки на систему. Некоторые методы, используемые при статической балансировке: перечисление и поиск в пространстве решений, теория графов, теория очередей.

При динамическом подходе, решения о балансировке нагрузки принимаются на основе текущего состояния системы; задачам разрешается динамически перемещаться с перегруженного узла на недостаточно загруженный узел для более быстрой обработки. Способность реагировать на изменения в системе является основным преимуществом динамического подхода к балансировке нагрузки. Хотя найти динамическое решение намного сложнее, чем статическое, динамическая балансировка нагрузки может повысить производительность за счет своей адаптивности. Далее сосредоточимся именно на таких подходах.

Механизм управления динамической балансировкой может быть распределенным, когда ответственность за балансировку распределяется между узлами, и нераспределенным, когда от-

ветственность за балансировку лежит на одном или нескольких узлах; включает централизованный (один центральный узел управляет балансировкой всей системы) и полу распределенный (узлы делятся на кластеры с центральными узлами для балансировки внутри кластера) подходы. [1]

2.2 Обзор существующих методов

Рассмотрим методы балансировки нагрузки, применимые к распределённым вычислительным системам с акцентом на гибридные конфигурации (платформы типа BOINC + стабильные кластерные сегменты). Среди критериев для них можно выделить: простота мониторинга, устойчивость к churn (неконтролируемое подключение / отключение узлов), гетерогенность ресурсов, величина накладных расходов, масштабируемость. Всего можно выделить несколько групп подходов к балансировке нагрузки:

- Классические статические методы (Round Robin, Random Strategy) — просты в мониторинге, однако не учитывают размер и время выполнения задач
- Классические динамические методы (Least-Connection) — также просты в мониторинге, но не учитывает общую нагрузку на узлы
- Адаптивные методы — балансировка происходит на основе метрик, вычисляемых в реальном времени; требуют постоянных вычислений
- Предиктивные ML подходы — планировщики, которые хорошо работают даже в сложных условиях, но требуют времени и данных для обучения модели [5]
- Существующие решения для BOINC (CluBoRun) — механизмы верификации, учета доступности, отслеживания churn и другие [3]

Большинство классических и оптимизационных алгоритмов работают при допущении, что узлы доступны и их характеристики относительно стабильны. В средах типа BOINC это допущение неприменимо: изменчивость системы и разнородность устройств приводит к частому перераспределению задач и дополнительным затратам. Если в гомогенных системах перенос задач — относительно стандартный процесс, в гибридных системах такой подход может привести к серьёзному росту накладных расходов из-за неоднородности устройств и отсутствия preemptive migration. Также нужно отметить, что для гибридных систем важно сохранять баланс между количеством собираемых метрик и их полезности для планировщика, что труднодостижимо при подходах с глобальным мониторингом узлов. [4]

2.3 Особенности BOINC

Платформа BOINC использует модель *client-initiated*: вычислительный узел сам запрашивает работу через RPC. Scheduler сканирует shared-memory массив feeder, оценивает совместимость workunit с host и ранжирует кандидатов по score. Назначенная задача не мигрирует на другой host до timeout или повторной репликации. Поле `rsc_frops_est` задаёт оценку сложности, которая может расходиться с реальным временем выполнения — это важный источник ошибок для любой adaptive-политики. Решение CluBoRun [3] интегрирует кластерные ресурсы в BOINC-проект, но не решает глобальную балансировку по классам устройств (кластер / desktop / mobile).

2.4 Сравнение подходов

Поймем, насколько применимы указанные подходы к гибридным системам и выделим их недочёты:

Группа методов		Тип	Требования мониторинга	Цель подхода	Минусы
Классические статические		Простые	Низкие	Простое планирование	Не учитывают размер задач
Классические динамические		Реактивные	Низкие/средние	Сократить число одновременных задач на узле	Не учитывает CPU-time узла
Адаптивные методы		Feedback-driven	Средние/высокие	Низкая задержка и оптимальная нагрузка на узлы	Высокие накладные расходы
Предиктивные подходы	ML-	Обучаемые	Высокие	Минимизация целевой функции ошибки	Требуют данных и обучения
CluBORun		Интеграция кластера / устройств	Низкие/средние	Использование idle ресурсов, учёт доступности	Не решает глобальную балансировку

2.5 Выводы из обзора

- На данный момент существует широкий набор методов балансировки, однако немногие работы систематически исследуют их поведение в гибридных системах
- Основные пробелы: недостаточная оценка влияния churn, упрощённое моделирование сетевых затрат при перераспределении задач, отсутствие оценки накладных расходов мониторинга и планировщика

Данная работа направлена на устранение выявленных ограничений за счёт оптимизации накладных расходов планировщика, более точного учёта динамической доступности узлов, снижения частоты миграций задач и повышения общей производительности системы при рациональном использовании вычислительных ресурсов.

Список литературы

- [1] Ali M. Alakeel. «A Guide to Dynamic Load Balancing in Distributed Computer Systems». B: *International Journal of Computer Science and Network Security* 1.2 (2010).
- [2] Tony Bourke. *Server Load Balancing*. O'Reilly & Associates, 2001.
- [3] Dolgov A. Kurochkin I. «Load balancing in grid systems from personal devices on the BOINC platform». B: *XIV All-russian multi-conference on management issues of the MCPU-2021*. 2021.
- [4] Jerry Gao Mazedur Rahman Samira Iqbal. «Load Balancer as a Service in Cloud Computing». B: *IEEE 8th International Symposium on Service Oriented System Engineering (SOSE)*. 2014.
- [5] Lun WANG. «Low-Latency, High-Throughput Load Balancing Algorithms». B: *Journal of Computer Technology and Applied Mathematics* 10.6 (2024), с. 153—160.